Nome: Vincent Ka Soen Lam RA: 2161392613004

Relatório: Teoria dos Conjuntos nos Códigos

1. Onde a teoria dos conjuntos aparece

`main.cob` (COBOL). Todas as operações foram implementadas à mão, com laços (`PERFORM VARYING`) e condicionais (`IF` / `EVALUATE`):

- **Pertinência (x ∈ A):** o parágrafo `PERTENCE` percorre a tabela; é a base de todas as outras operações.
- **União, interseção e diferenças:** A ∪ B copia A e insere de B só o que falta; A ∩ B guarda x ∈ A com x ∈ B; A − B guarda x ∈ A com x ∉ B; A △ B = (A − B) ∪ (B − A).
- **Complemento:** x ∈ U e x ∉ A.
- **Inclusão (⊆ e ⊂):** todo x de A pertence a B; a inclusão própria exige também |A| < |B|.
- **Cardinalidade e casos especiais:** contador de elementos, conjunto vazio, iguais, disjuntos e a distinção entre `{}` , `0` e ∅.

`conjuntos_numericos.py` (Python). Usa a hierarquia $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$, com $\mathbb{I} \subset \mathbb{R}$ e $\mathbb{R} = \mathbb{Q} \cup \mathbb{I}$, para classificar cada número e justificar a pertinência. O crivo de Eratóstenes e os divisores geram subconjuntos de $\mathbb{N}$.

`controle_acesso.py` e cenários 1 a 3 (Python). Cada perfil é um conjunto de permissões, e as permissões de um usuário são a união dos seus perfis. `pode` é pertinência, `contido` e `contido_proprio` são ⊆ e ⊂, e `quem` é um conjunto definido por compreensão. Os cenários testam pertinência e união, hierarquia de perfis e casos especiais (iguais, disjuntos, vazio).

2. Parte mais difícil: o programa em COBOL

A parte mais difícil foi o programa em COBOL, porque a linguagem não possui estrutura adequada para manipular conjuntos e suas funções. Em Python, `set` , `in` , `|` , `&` , `-` e `<=` resolvem cada operação em uma linha; em COBOL foi preciso construir tudo do zero:

- **Sem tipo conjunto nem lista dinâmica:** cada conjunto virou uma tabela de tamanho fixo com contador, com limites de 20 elementos (A e B) e 40 (U e resultado, pelo pior caso da união).
- **Sem operadores de conjunto:** cada operação foi reescrita a partir da definição matemática, com laços aninhados e chamadas a `PERTENCE` .
- **Sem controle de repetição:** a inserção verifica a pertinência antes de gravar; um teste mostrou o repetido consumindo uma vaga, e isso foi corrigido.

- **Sem valor nulo:** o elemento ∅ usa um valor reservado (99999), e a entrada é validada manualmente (±9999).

Por isso o COBOL ficou com 1058 linhas, contra 85 e 59 nos programas em Python.